

# INFORME

## Refactorización con Patrones GRASP – Sistema de Gestión de Clínica Odontológica

*Proceso de Desarrollo de Software – UADE 2026*

**Integrantes:** Daher, Ezequiel · Quesada, Bautista · Claros, Alex

## 1. Introducción

El presente informe documenta la revisión y refactorización del sistema de gestión para una clínica odontológica, tomando como criterio de análisis los patrones GRASP de asignación de responsabilidades. El sistema, desarrollado en Java con interfaz gráfica Swing y persistencia en archivos de texto, administra pacientes, odontólogos, secretarías y turnos.

El trabajo partió de una versión funcional. El software cumplía con los requisitos solicitados: permitía registrar los cuatro tipos de entidad, agendar turnos, consultarlos y calcular el monto a facturar. Sin embargo, una revisión del diseño interno reveló que varias responsabilidades estaban asignadas a clases que no correspondían según los criterios de GRASP.

El diagnóstico central fue el siguiente: el módulo de Turno ya aplicaba un diseño correcto, mientras que los módulos de Paciente, Odontólogo y Secretaria habían quedado en un estado anterior. En Turno, la creación de instancias ocurría en el servicio después de validar, el controlador era pura delegación, y la validación estaba separada de la consulta. En los otros tres módulos, en cambio, la creación se resolvía en el controlador o directamente en la vista, y la actualización se hacía con métodos set invocados desde afuera de la entidad.

Esta asimetría resultó ser la explicación de fondo de la mayoría de los problemas detectados. Por ese motivo, la estrategia adoptada no fue diseñar una arquitectura nueva, sino replicar en los tres módulos rezagados el diseño que el módulo de Turno ya tenía. Esta decisión redujo el riesgo del refactor y permitió sostener cada corrección mostrando que ya existía un precedente funcionando dentro del mismo sistema.

Se relevaron quince problemas de asignación de responsabilidades, identificados como P-01 a P-15. Los quince fueron corregidos y verificados.

## 2. Metodología de Trabajo

El procedimiento aplicado a cada hallazgo constó de cuatro pasos. Primero se identificó la responsabilidad mal ubicada, determinando qué clase estaba haciendo algo que no le correspondía. Luego se determinó a quién debía pertenecer, aplicando Experto en Información para las responsabilidades de conocer y calcular, Creador para las de instanciar, y Controlador para las de recibir eventos de la interfaz. En tercer lugar se evaluó la alternativa elegida,

contrastándola contra Bajo Acoplamiento y Alta Cohesión para confirmar que el rebalanceo mejoraba efectivamente el diseño y no se limitaba a mover el problema de lugar. Finalmente se implementó el cambio, se recompiló el sistema completo y se ejecutó la batería de pruebas.

### 3. Resumen de Hallazgos

La siguiente tabla resume los quince problemas detectados, los patrones que sustentan cada corrección y las clases donde puede verificarse el resultado.

| ID   | Problema detectado                          | Patrones aplicados                          | Dónde verlo                              |
|------|---------------------------------------------|---------------------------------------------|------------------------------------------|
| P-01 | Se modificaba la entidad antes de validar   | Controlador · Experto · Alta Cohesión       | Controllers, services y actualizarDatos  |
| P-02 | Baja de actores con turnos asociados        | Experto · Bajo Acoplamiento                 | HistorialTurnos y eliminar               |
| P-03 | La vista creaba odontólogos                 | Creador · Controlador · Bajo Acoplamiento   | OdontologoPanel y OdontologoServiceImpl  |
| P-04 | Los controladores creaban entidades         | Creador · Controlador · Alta Cohesión       | Controllers y services de actores        |
| P-05 | Consultas resueltas fuera del repositorio   | Experto · Alta Cohesión                     | ITurnoRepository y TurnoRepository       |
| P-06 | Dependencias a implementaciones concretas   | Bajo Acoplamiento                           | Interfaces de service y repository       |
| P-07 | El servicio construía sus colaboradores     | Creador · Bajo Acoplamiento · Alta Cohesión | DependenciasClinica y TurnoServiceImpl   |
| P-08 | MainFrame conocía la composición completa   | Controlador · Bajo Acoplamiento             | Main y MainFrame                         |
| P-09 | La persistencia manipulaba contadores de ID | Creador · Bajo Acoplamiento                 | Métodos rehidratar y clases Persistencia |

| ID   | Problema detectado                     | Patrones aplicados                          | Dónde verlo                           |
|------|----------------------------------------|---------------------------------------------|---------------------------------------|
| P-10 | Combos con Strings y listas paralelas  | Experto · Bajo Acoplamiento                 | OpcionEntidad y TurnoFormularioPanel  |
| P-11 | Invariantes validadas desde afuera     | Experto · Alta Cohesión                     | Persona, Domicilio, Paciente y Turno  |
| P-12 | La facturación desarmaba el turno      | Experto · Bajo Acoplamiento · Alta Cohesión | Facturador y Turno.calcularMonto      |
| P-13 | TurnoValidador mezclaba tareas         | Alta Cohesión · Bajo Acoplamiento           | Validador, Resolutor y Disponibilidad |
| P-14 | TurnoPanel concentraba la presentación | Alta Cohesión                               | Componentes view/Turno                |
| P-15 | Turnos inactivos bloqueaban la agenda  | Experto en Información                      | EstadoTurno, Turno y TurnoRepository  |

## 4. Correcciones Implementadas

### P-01 – Se modificaba la entidad antes de validar

**Situación previa.** Los controladores de Paciente, Odontólogo y Secretaria recuperaban la instancia guardada en el repositorio y ejecutaban sobre ella una secuencia de métodos set (setNombre, setApellido, setDni, entre otros) antes de que el servicio comprobara duplicados y reglas de negocio. El efecto era que, si la validación fallaba, los cambios ya aplicados no se revertían. Como los repositorios guardan referencias en un HashMap, la entidad quedaba parcialmente modificada en memoria y la operación no era atómica.

**Patrones aplicados.** Por Controlador, el controlador debe limitarse a recibir el evento de la interfaz y delegarlo, ya que modificar el estado del dominio excede esa función. Por Experto en Información, la entidad es quien posee sus propios datos y debe ser ella la que aplique el cambio, mientras que el servicio es quien conoce las reglas de unicidad del repositorio. Por Alta Cohesión, el controlador estaba mezclando tres tareas sin relación entre sí: coordinar la llamada, editar la entidad y disparar la validación.

**Cambio aplicado.** Los controladores pasaron a enviar un objeto de transferencia de edición (PacienteEdicion, OdontologoEdicion, SecretariaEdicion). Cada servicio valida la totalidad de

los datos y verifica duplicados antes de invocar a la entidad. Recién entonces la entidad aplica el cambio completo mediante un único método `actualizarDatos`, que valida y modifica en una sola operación.

**Dónde verlo.** `PacienteController.actualizarPaciente`, que quedó reducido a pura delegación; `PacienteServiceImpl.actualizar`, que valida, busca, comprueba unicidad y recién ahí aplica; y `Paciente.actualizarDatos`. La prueba `comprobarActualizacionAtomica` intenta asignar a un paciente el DNI de otro y confirma que, tras el rechazo, el nombre original permanece intacto.

## **P-02 – Baja de actores con turnos asociados**

**Situación previa.** Paciente y Odontólogo bloqueaban la eliminación únicamente si existían turnos futuros activos, mediante `tieneTurnosFuturos`. Secretaria no verificaba ninguna relación antes de eliminar. Esto permitía dar de baja actores que todavía estaban referenciados por turnos cancelados, completados o pasados, de modo que al recargar el sistema desde los archivos esos turnos podían quedar apuntando a registros inexistentes.

**Patrones aplicados.** Por Experto en Información, `HistorialTurnos` es la clase que contiene la colección de turnos del actor y por lo tanto la que tiene la información necesaria para responder si existe algún turno asociado. Por Bajo Acoplamiento, los servicios consultan una operación del actor sin necesidad de conocer cómo está estructurado el historial internamente, de modo que un cambio en esa estructura no los afecta.

**Cambio aplicado.** Se incorporó el método `tieneTurnos` en `HistorialTurnos` y se lo expuso a través de `Paciente`, `Odontologo` y `Secretaria`. Los tres servicios bloquean ahora la baja física mientras exista cualquier turno asociado, sin importar su estado ni su fecha. La política preserva el historial clínico y evita referencias rotas.

**Dónde verlo.** `HistorialTurnos.tieneTurnos`, los métodos `tieneTurnos` de las tres entidades de actor, y el método `eliminar` de los tres servicios. La prueba crea un turno e intenta eliminar la secretaria que lo registró, verificando que la baja sea rechazada.

## **P-03 – La vista creaba odontólogos**

**Situación previa.** `OdontologoPanel` conocía la clase `OdontologoFactory` y construía directamente la subclase de odontólogo antes de llamar al controlador, resolviendo la selección con un switch sobre el índice del combo de especialidad. De este modo, una clase de la capa de presentación quedaba acoplada a la estrategia de creación del dominio y tomaba decisiones que exceden la captura de datos.

**Patrones aplicados.** Por Creador, el servicio es quien tiene los datos de inicialización ya validados y quien registra la instancia en el repositorio, dos condiciones que lo señalan como creador apropiado; la vista no cumple ninguna de ellas. Por Controlador, la interacción debe seguir el recorrido vista, controlador, dominio, y era justamente ese paso intermedio el que se

estaba salteando. Por Bajo Acoplamiento, OdontologoPanel dejó de depender de la fábrica y de los detalles de construcción de las subclases.

**Cambio aplicado.** Se crearon los objetos de transferencia OdontologoRegistro y OdontologoEdicion. La vista arma el objeto a partir de los campos del formulario, y OdontologoServiceImpl valida los datos y recién entonces invoca a la fábrica. La fábrica conserva la selección de la subclase según la especialidad, pero su cliente pasó a ser el servicio en lugar de la pantalla.

**Dónde verlo.** OdontologoPanel.guardar, que arma el objeto de transferencia y llama al controlador; OdontologoServiceImpl.registrar, que valida, comprueba la matrícula y crea; y OdontologoFactory.crear. La compilación confirma que la vista ya no importa la fábrica ni ninguna subclase concreta de odontólogo.

#### **P-04 – Los controladores creaban entidades de dominio**

**Situación previa.** PacienteController construía instancias de Paciente y de Domicilio, y SecretariaController construía instancias de Secretaria. Los métodos registrarPaciente y registrarSecretaria contenían expresiones new y métodos auxiliares de construcción, de manera que los controladores conocían los constructores y los detalles de composición de las entidades cuando su función es únicamente recibir el evento y delegarlo.

**Patrones aplicados.** Por Creador, los servicios poseen los datos ya validados y son quienes registran las instancias en el repositorio. Por Controlador, los controladores quedaron reducidos a adaptadores del evento de interfaz. Por Alta Cohesión, la creación, la validación de unicidad y el registro son responsabilidades relacionadas entre sí, y ahora conviven en el servicio en lugar de estar repartidas entre dos capas.

**Cambio aplicado.** PacienteServiceImpl crea el Domicilio y el Paciente, y SecretariaServiceImpl crea la Secretaria. Los controladores se limitan a trasladar el objeto de transferencia. Los cuatro flujos de alta siguen ahora la misma dirección: vista, objeto de transferencia, controlador, servicio, repositorio.

**Dónde verlo.** PacienteController y SecretariaController, que no contienen una sola expresión new; PacienteServiceImpl.registrar y SecretariaServiceImpl.registrar.

#### **P-05 – Consultas resueltas fuera del repositorio**

**Situación previa.** TurnoConsultaService solicitaba la colección completa mediante listarTodos y luego filtraba y ordenaba los resultados con streams, a través de un método propio filtrarYOrdenar. El servicio necesitaba entonces conocer la colección entera y replicaba criterios de búsqueda que son propios del componente que administra los turnos.

**Patrones aplicados.** Por Experto en Información, TurnoRepository es quien posee la colección completa de turnos y por lo tanto el experto natural para resolver búsquedas sobre ella. Por

Alta Cohesión, quedó una división clara de tareas: el servicio valida los parámetros de entrada y coordina, mientras el repositorio filtra y ordena.

**Cambio aplicado.** `ITurnoRepository` declara consultas específicas (`listarPorPaciente`, `listarPorOdontologo`, `listarPorSecretaria`, `listarPorEstado`, `buscarPorRangoFechas` y `buscarPorOdontologoYPaciente`) y `TurnoRepository` las implementa apoyándose en un único método privado `filtrarYOrdenar` interno. `TurnoConsultaService` valida la existencia de los actores o la coherencia de los rangos de fecha y delega la búsqueda.

**Dónde verlo.** La interfaz `ITurnoRepository` como contrato de consultas, `TurnoRepository` donde `filtrarYOrdenar` pasó a ser privado, y `TurnoConsultaService`. La prueba verifica las consultas por paciente, por odontólogo, por secretaria y por estado.

## **P-06 – Dependencias sobre implementaciones concretas**

**Situación previa.** Los controladores y los servicios declaraban sus campos y parámetros de constructor usando tipos concretos: `PacienteController` dependía de `PacienteServiceImpl`, y `TurnoServiceImpl` junto con `TurnoValidador` dependían de las clases `Repository` concretas. Las interfaces genéricas que ya existían en el proyecto no funcionaban como límites efectivos porque los consumidores no las usaban, de modo que sustituir una implementación obligaba a modificar a todos sus clientes.

**Patrones aplicados.** Por Bajo Acoplamiento, que es el patrón que describe textualmente este problema: una clase con alto acoplamiento confía en muchas otras, y un cambio en una clase relacionada fuerza cambios locales. Al depender de contratos pequeños y estables en lugar de implementaciones, el impacto de un cambio queda contenido detrás de la interfaz.

**Cambio aplicado.** Se incorporaron los contratos `PacienteService`, `OdontologoService`, `SecretariaService` y `TurnoService`, junto con las cuatro interfaces específicas de repositorio. Las clases con sufijo `Impl` implementan esos contratos, y las implementaciones concretas se seleccionan en un único lugar del sistema, que es `DependenciasClinica`.

**Dónde verlo.** Los contratos de servicio y de repositorio, los campos de los cuatro controladores y los constructores de los servicios. Una búsqueda sobre el código confirma que no queda ninguna referencia a clases con sufijo `Impl` ni a repositorios concretos fuera de la clase de configuración y de la capa de persistencia.

## **P-07 – El servicio construya sus propios colaboradores**

**Situación previa.** El constructor de `TurnoServiceImpl` ejecutaba internamente las instanciaciones de `TurnoValidador` y `TurnoConsultaService`. El servicio recibía los repositorios por parámetro, pero decidía por su cuenta qué colaboradores usar y cómo construirlos, sumando así el ensamblado a su responsabilidad de coordinar el caso de uso.

**Patrones aplicados.** Por Creador, `DependenciasClinica` es la raíz de composición del sistema, la clase que tiene los datos de inicialización de los colaboradores y que los utiliza

estrechamente, lo que la señala como su creadora natural. Por Bajo Acoplamiento, TurnoServiceImpl recibe objetos ya configurados y no conoce cómo se construyen. Por Alta Cohesión, el servicio dejó de ocuparse de una tarea sin relación con coordinar turnos.

**Cambio aplicado.** DependenciasClinica crea TurnoValidador, TurnoResolutor, DisponibilidadTurnos y TurnoConsultaService, y los inyecta por constructor. El constructor de TurnoServiceImpl quedó reducido a asignar las cinco dependencias recibidas, sin contener ninguna expresión new.

**Dónde verlo.** DependenciasClinica, que es el único lugar del sistema donde se instancian colaboradores, y el constructor de TurnoServiceImpl. La prueba arma una composición alternativa en memoria e inyecta los mismos contratos, lo que confirma que el servicio es independiente de cómo se lo ensambla.

## **P-08 – MainFrame conocía la composición completa**

**Situación previa.** MainFrame recibía el objeto DependenciasClinica completo, extraía de él los controladores mediante sus getters, y llamaba directamente a guardarDatos al cerrarse la ventana. La ventana dependía así del contenedor entero de dependencias y conocía además una operación de infraestructura que pertenece a la capa de persistencia.

**Patrones aplicados.** Por Controlador, son los controladores los que actúan como fachada del dominio hacia la interfaz, de modo que la ventana debe administrar la interacción visual y no resolver dependencias ni invocar servicios de infraestructura. Por Bajo Acoplamiento, la vista pasó de conocer el contenedor completo a conocer únicamente los cuatro controladores que efectivamente usa, más una acción de guardado.

**Cambio aplicado.** Main construye el MainFrame pasándole los cuatro controladores y una referencia al método de guardado. MainFrame almacena esa acción como un Runnable, sin saber qué hay detrás de ella ni que existe una capa de persistencia.

**Dónde verlo.** Main.main, el constructor de MainFrame y el método configurarGuardadoAlCerrar. No queda ningún import de la clase de configuración en toda la capa de vista.

## **P-09 – La persistencia manipulaba los contadores de ID**

**Situación previa.** Cada clase de persistencia, al reconstruir una entidad desde el archivo, ejecutaba una secuencia que fijaba el contador estático al valor anterior al identificador leído, creaba la instancia, y luego volvía a ajustar el contador al máximo. La reconstrucción dependía por lo tanto de estado estático público, del orden en que se leían las líneas del archivo, y de detalles internos de cómo cada entidad genera su identidad.

**Patrones aplicados.** Por Creador, cada entidad, o su fábrica en el caso de Odontólogo, es quien tiene los datos de inicialización y quien sabe cómo reconstruirse a partir de ellos. Por

Bajo Acoplamiento, desaparecieron las llamadas desde la capa de persistencia hacia métodos estáticos que exponían el mecanismo interno de identidad.

**Cambio aplicado.** Se incorporaron las operaciones de rehidratación en Paciente, Secretaria, Turno y OdontologoFactory. La persistencia entrega el identificador leído del archivo y la entidad se encarga internamente de actualizar su contador, sin que nadie desde afuera manipule ese estado.

**Dónde verlo.** Los métodos rehidratar de las entidades, OdontologoFactory.rehidratar y las clases de persistencia. La prueba rehidrata un paciente con identificador 500 y luego crea uno nuevo, confirmando que este último recibe un identificador mayor. Una búsqueda sobre el código confirma que el método setContadorId ya no existe en el sistema.

## **P-10 – Combos con Strings y listas paralelas**

**Situación previa.** Los combos de la pantalla de turnos almacenaban cadenas de texto, y TurnoPanel mantenía en paralelo tres listas cuyo índice debía coincidir exactamente con el índice visual del combo, resolviendo la correspondencia mediante `getSelectedIndex`. Cualquier cambio en el orden o en el formato de las etiquetas podía asociar el turno a una entidad distinta de la seleccionada. El estado del turno, además, se reconstruía a partir del texto mostrado en pantalla.

**Patrones aplicados.** Por Experto en Información, cada opción tipada conserva directamente la referencia a la entidad y es quien responde por su identificador. Por Bajo Acoplamiento, la selección dejó de depender del formato de la etiqueta visible y de la sincronización entre listas paralelas.

**Cambio aplicado.** Se creó la clase genérica `OpcionEntidad`, que envuelve la entidad junto con su etiqueta de presentación y expone el identificador. `TurnoFormularioPanel` utiliza combos de opciones tipadas y un combo de `EstadoTurno` para el estado, de modo que el método `toString` solo controla la etiqueta visible y no interviene en la selección de dominio.

**Dónde verlo.** `OpcionEntidad` y el método `leerRegistro` de `TurnoFormularioPanel`. No queda ningún combo de cadenas en el sistema, ni ninguna conversión de texto a `EstadoTurno`.

## **P-11 – Invariantes locales validadas desde afuera**

**Situación previa.** Los servicios extraían los datos de las entidades mediante getters, ejecutaban las validaciones por fuera y luego aplicaban los cambios con métodos `set`. Las entidades, por su parte, aceptaban datos en el constructor sin comprobarlos y permitían modificaciones parciales. El resultado era que reglas que dependen exclusivamente de los atributos propios de un objeto no estaban protegidas por el objeto que posee esa información.

**Patrones aplicados.** Por Experto en Información, `Persona`, `Domicilio`, `Paciente`, `Odontologo` y `Turno` son quienes tienen la información necesaria para validar sus propias invariantes. Por Alta Cohesión, quedó una separación clara: los servicios conservan las reglas que requieren

consultar el repositorio, como la unicidad de DNI o de matrícula, y las entidades conservan su consistencia interna.

**Cambio aplicado.** Persona y Domicilio validan sus datos en el constructor. Paciente, Odontologo, Secretaria y Turno ofrecen operaciones cohesivas de actualización que validan antes de modificar, y dejaron de exponer métodos set generales para datos personales o para la edición completa del turno.

**Dónde verlo.** El constructor de Persona junto con validarDatosPersonales y actualizarDatosPersonales, el constructor de Domicilio, Paciente.actualizarDatos y Turno.actualizar. La prueba de actualización atómica descrita en el primer hallazgo ejercita directamente estas invariantes.

## **P-12 – La facturación desarmaba el turno**

**Situación previa.** TurnoConsultaService extraía del turno las referencias al Paciente y al Odontologo y se las entregaba por separado a Facturador, cuya firma recibía ambas entidades. Esto obligaba al llamador a desarmar el turno para poder calcular su monto, cuando el turno ya conoce a ambos participantes, de modo que la navegación resultante era innecesaria.

**Patrones aplicados.** Por Experto en Información, el Turno tiene acceso directo al paciente y al odontólogo involucrados en el cálculo y es por lo tanto el experto para responder por su monto. El razonamiento se encadena en cascada: el turno le pregunta al paciente, el paciente consulta su cobertura, y la cobertura aplica la política correspondiente según la tarifa del odontólogo. Por Bajo Acoplamiento, Facturador pasó de conocer dos clases del dominio a conocer solamente una. Por Alta Cohesión, TurnoConsultaService se limita ahora a localizar el turno y delegar el cálculo.

**Cambio aplicado.** Facturador recibe el Turno completo e invoca calcularMonto sobre él. El turno delega en el paciente, y el paciente a su vez delega la política de cobro en CoberturaPaciente, donde cada tipo de cobertura define su propio cálculo. El recorrido completo va de TurnoConsultaService a Facturador, de allí a Turno, luego a Paciente y finalmente a CoberturaPaciente. Cada paso de esta cadena está justificado por Experto en Información, ya que en cada nivel la responsabilidad recae en la clase que tiene la información necesaria para resolverla.

**Dónde verlo.** Facturador.calcularMonto, Turno.calcularMonto, Paciente.calcularMonto y CoberturaPaciente. La prueba compara el monto de un paciente particular contra la tarifa base de su odontólogo y confirma que coinciden.

## **P-13 – TurnoValidador mezclaba responsabilidades**

**Situación previa.** TurnoValidador acumulaba tres tareas distintas: validaba los datos propios del turno, resolvía las referencias a las cuatro entidades relacionadas, y consultaba la existencia de conflictos en la agenda. Para poder hacerlo mantenía cuatro repositorios como

dependencias. La clase tenía por lo tanto tres motivos independientes de cambio: las reglas de los campos, la búsqueda de referencias y la disponibilidad horaria.

**Patrones aplicados.** Por Alta Cohesión, que describe exactamente este caso: una clase de baja cohesión hace demasiado trabajo no relacionado entre sí, lo que la vuelve difícil de entender, de reutilizar y de mantener, y la deja constantemente afectada por los cambios. Por Bajo Acoplamiento, como consecuencia directa de la separación, TurnoValidador pasó de depender de cuatro repositorios a no depender de ninguno: hoy no tiene atributos.

**Cambio aplicado.** Se repartieron las responsabilidades en cuatro clases con foco propio. TurnoValidador conserva las reglas de los datos del turno; TurnoResolutor se ocupa de buscar las entidades referenciadas; DisponibilidadTurnos controla los conflictos de agenda; y TurnoConsultaService coordina las consultas. TurnoServiceImpl expresa el caso de uso combinando estos colaboradores, que recibe ya contruidos.

**Dónde verlo.** TurnoValidador, que quedó sin atributos, junto con TurnoResolutor, DisponibilidadTurnos y TurnoConsultaService. El método registrarTurno de TurnoServiceImpl se lee hoy como la secuencia del caso de uso. La prueba arma e inyecta los cuatro colaboradores y ejecuta altas, consultas y cambios de estado.

#### **P-14 – TurnoPanel concentraba toda la presentación**

**Situación previa.** TurnoPanel reunía en una sola clase de 457 líneas la tabla, el formulario, los combos, los filtros, el parseo de fechas, los botones y la coordinación de eventos. Si bien todo ese contenido pertenece legítimamente a la capa de presentación, la clase tenía demasiados motivos internos de cambio y resultaba difícil de recorrer.

**Patrones aplicados.** Por Alta Cohesión, cuyo criterio es mantener manejable la complejidad. Al separar el formulario, la tabla y los filtros en componentes con responsabilidades específicas, cada uno queda enfocado en una sola cosa.

**Cambio aplicado.** Se extrajeron TurnoFormularioPanel, TurnoTablaPanel y TurnoFiltrosPanel, además de la clase auxiliar OpcionEntidad descrita anteriormente. TurnoPanel quedó como coordinador de eventos entre los controladores y los componentes visuales, reducido a 287 líneas.

**Dónde verlo.** TurnoPanel, que solo coordina, y los tres componentes extraídos. La compilación de la interfaz completa verifica las nuevas colaboraciones.

#### **P-15 – Turnos inactivos bloqueaban la agenda**

**Situación previa.** TurnoRepository consideraba que existía un conflicto ante cualquier coincidencia de odontólogo, fecha y hora, sin tener en cuenta el estado del turno. Como consecuencia, un turno cancelado o completado seguía ocupando su horario indefinidamente, y reactivar un turno previamente cancelado podía generar una superposición no detectada.

**Patrones aplicados.** Por Experto en Información, EstadoTurno es quien tiene la información sobre qué estados ocupan efectivamente la agenda, y Turno es quien conoce su propio estado actual, de modo que la pregunta debe dirigirse a ellos. Antes, esa información estaba implícita y repetida en el repositorio y en el servicio, que comparaban campos por su cuenta sin consultar al experto.

**Cambio aplicado.** Se incorporó el método ocupaAgenda en EstadoTurno, y el método homónimo en Turno que delega en su estado. El repositorio filtra ahora únicamente los turnos que ocupan agenda, y TurnoServiceImpl valida la disponibilidad también al reactivar o al editar un turno. De este modo, los turnos cancelados y completados liberan el horario, y una reactivación es rechazada si ese horario ya fue tomado.

**Dónde verlo.** EstadoTurno.ocupaAgenda, Turno.ocupaAgenda, los métodos de conflicto de TurnoRepository y TurnoServiceImpl.cambiarEstado. La prueba cancela un turno, registra otro en el mismo horario, y confirma que el primero ya no puede reactivarse.